Phase 1 Review Brief — Rebinding CTSM Data Access

What changed, what it accomplishes, and what it commits you to

`exsoil-nsf-prototype` `feature/arm64-multiarch-rebuild` `commit 430d416` `2026-07-30`

Phase 1 set out to let the analysis notebooks read simulation output from inside the container instead of from a private S3 bucket. Until now every read went through `campus.s3.wisc.edu` and required credentials, and nothing in the codebase could read the output that CTSM (the Community Terrestrial Systems Model, the land-surface model this project runs) actually produces today.

This brief covers what changed, why each piece exists, and what it commits you to. Unlike a pre-implementation brief, **both sides of every comparison are real code** — nothing here is illustrative. "Before" is the tree at `430d416^`, "after" is what is committed at `430d416`. Nothing is merged; the branch has no open pull request, so any of this can be reverted.

> **The one-sentence version:** the readers were looking for files whose names, directory, and binary format had all changed, and rather than swapping one convention for another, the new code discovers which convention it is looking at — because the old and new conventions both have to keep working.

Reading order

1. Discover the stream name instead of hardcoding it — the core fix
2. Probe the archive layouts — depends on 1
3. Turn discovery into a file-list primitive that fails loudly — depends on 1, 2
4. Choose the file reader from the file's magic number — independent
5. The source-agnostic entry point — depends on 3, 4
6. Revive the dead local branch in the soil-profile plot — depends on 3
7. Parameterise the stream token on the two S3 readers — independent
8. Tests at the reader boundary — depends on all

The changes

CHANGE 1 OF 8

Discover the stream name instead of hardcoding it

data_access.py:249

BEFORE · DATA_ACCESS.PY:180

```
fname_prefix = f"{neon_site}.transient.clm2.h1.{year}"
```

AFTER · DATA_ACCESS.PY:249

```
STREAM_TOKENS = {
    "daily":   ("h1a", "h1"),
    "monthly": ("h0a", "h0"),
}
```

WHAT IT DOES, PLAINLY

CTSM writes its results into many files whose names encode which "stream" they belong to — a stream being one bundle of variables written at one frequency. The code used to hardcode the single token `h1`. It now holds an ordered list per frequency and tries each against the disk, newest naming first, taking whichever matches.

WHY IT MATTERS TO THE DOMAIN

CTSM 5.4 renamed its streams: daily averages moved from `h1` to `h1a` and monthly from `h0` to `h0a`. Every file the container produces carries the new names, so the old pattern matched **zero of 5,415 live files**. No simulation output could reach any notebook. Meanwhile the reference copies used to validate results are from an older model generation and still carry the old names, so both conventions have to work at once — this is not a migration that finishes.

WHY IT MATTERS TO THE CODEBASE

Probing rather than configuring means no caller has to know which generation produced the data it is reading, and no config flag has to be threaded through the notebooks. The cost is that a directory holding both conventions resolves to the newer one by list order rather than by anything explicit.

WATCH OUT FOR

Order is load-bearing: `"h1a"` must stay first in the tuple. Reversing it would silently prefer legacy files wherever both exist. A test asserts this, but it is the kind of thing a tidy-up could undo.

CHANGE 2 OF 8

Probe the archive layouts rather than assuming one

data_access.py:269

BEFORE · DATA_ACCESS.PY:179

```
sim_path = f"archive_1/{neon_site}.transient/lnd/hist/"
```

AFTER · DATA_ACCESS.PY:279

```
fixed = [
    root / "lnd" / "hist",                      # root is already an archive
    root / "archive" / "lnd" / "hist",          # run_tower
    root / "archive" / case / "lnd" / "hist",   # S3 shape, staged locally
    root / case / "lnd" / "hist",
    root / "archive" / "archive" / case / "lnd" / "hist",   # reference copies
]
# run_neon_v2 writes archive/<site>/<control|VAR_VALUE>/lnd/hist
globbed = sorted(root.glob(f"archive/{neon_site}/*/lnd/hist"))
```

WHAT IT DOES, PLAINLY

Instead of one hardcoded directory, the code checks a short list of known locations and uses the first that contains matching files.

WHY IT MATTERS TO THE DOMAIN

Where results land depends on which wrapper ran the simulation. `run_tower` archives a single case flat; `run_neon_v2.py`, which is what the Hubs drive, inserts a site segment and an experiment segment so a perturbed run and its control stay separate. A scientist should not have to know which tool wrote their data in order to open it.

WHY IT MATTERS TO THE CODEBASE

This absorbs a real inconsistency instead of propagating it to every caller. The tradeoff is that adding a layout means editing this list, and a wrong-but-populated directory earlier in the list would win over the right one later.

WATCH OUT FOR

The `run_neon_v2` root derives from `dirname(base_case_root)`, *not* from `output_root` — the two are not interchangeable, and assuming they are is what produced the wrong path in the data contract.

CHANGE 3 OF 8

A file-list primitive that fails loudly

data_access.py:291

BEFORE · NO EQUIVALENT EXISTED

```
(callers globbed inline, or got an empty
 list back with no explanation)
```

AFTER · DATA_ACCESS.PY:332

```
raise FileNotFoundError(
    f"No CTSM {stream} history files for site={neon_site}, "
    f"year={year if year is not None else 'any'} under {root}.\n"
    "Tried:\n  " + "\n  ".join(tried)
)
```

WHAT IT DOES, PLAINLY

One function returns the sorted list of matching files. When it finds nothing it raises, naming every directory and pattern it attempted.

WHY IT MATTERS TO THE DOMAIN

The dangerous failure is not a crash, it is silence: an empty match flows downstream and reads as "this site has no data for that year", which is a plausible scientific statement and a false one. Someone can lose an afternoon to that.

WHY IT MATTERS TO THE CODEBASE

Returning paths rather than a Dataset matters, because the evaluation helpers in `neon_eval_utils` consume file lists. A Dataset-only interface would have left Hubs 2 and 3 keeping their own globs, which is most of what Phase 1 was trying to eliminate.

CHANGE 4 OF 8

Choose the reader from the file's magic number

data_access.py:339

BEFORE · DATA_ACCESS.PY:161

```
engine: str = "scipy",   # NetCDF-3 reader (CDF\x01/CDF\x02)
```

AFTER · DATA_ACCESS.PY:349

```
with open(path, "rb") as handle:
    magic = handle.read(4)
return "scipy" if magic in (b"CDF\x01", b"CDF\x02") else "netcdf4"
```

WHAT IT DOES, PLAINLY

Reads the first four bytes of a file to identify its format, then picks the matching reader library.

WHY IT MATTERS TO THE DOMAIN

NetCDF is the standard container format for climate data, and it has several on-disk variants. Live output is **CDF-5** (`b'CDF\x05'` , verified on disk), a 64-bit variant that the `scipy` reader cannot open at all. The reference copies are CDF-2, which it can. The plan guessed the live files were NetCDF-4 and suggested `h5netcdf` as the remedy; that would also have failed, because CDF-5 is not HDF5.

WHY IT MATTERS TO THE CODEBASE

Sniffing means no per-source configuration and no guessing when a third format appears. The `scipy` branch is kept rather than always using `netcdf4` because the S3 path feeds fsspec file objects, which the netcdf4 engine cannot consume.

CHANGE 5 OF 8

The source-agnostic entry point

data_access.py:353, 386, 394

BEFORE · ONLY AN S3 READER EXISTED

```
open_ctsm_hist_from_s3(input_label, s3_client,
                       bucket_name, neon_site, year, ...)
```

AFTER · DATA_ACCESS.PY:394

```
if resolve_source(source) == "s3":
    return open_ctsm_hist_from_s3(...)
return open_ctsm_hist_local(
    neon_site, year, output_root=output_root, stream=stream,
    input_label=input_label, **kwargs
)
```

WHAT IT DOES, PLAINLY

One call, `open_ctsm_hist(site, year)`, works whichever source the data is in. It resolves to local unless told otherwise via the `CTSM_DATA_SOURCE` environment variable or an explicit argument.

WHY IT MATTERS TO THE DOMAIN

A researcher pulling the container can open simulation output with no credentials and no setup. Reaching the shared S3 fixtures becomes the deliberate exception rather than the default path.

WHY IT MATTERS TO THE CODEBASE

The S3 functions are untouched and still work, so this adds a layer rather than replacing one. Credentials were already requested lazily, so nothing had to change to make them optional — the plan's Task 4 was effectively already done in the library.

WATCH OUT FOR

`CTSM_OUTPUT_ROOT` defaults to `/home/user`, which is right inside the container and wrong on a development host, where it must be set explicitly. The plan never pinned this default, so it is a decision made here rather than one handed down.

CHANGE 6 OF 8

Revive the dead local branch in the soil-profile plot

data_access.py:470

BEFORE · DATA_ACCESS.PY:280–286

```
sim_path = f"s3://clm-demonstration/archive_1/{neon_site}.transient/lnd/hist/"
...
is_s3 = isinstance(sim_path, str) and sim_path.startswith("s3://")
```

AFTER · DATA_ACCESS.PY:470

```
is_s3 = resolve_source(source) == "s3"
```

WHAT IT DOES, PLAINLY

The function decided whether it was reading S3 by testing a string it had assigned itself four lines earlier. That string always began with `s3://`, so the test was always true and the local branch beneath it — around sixty lines — had never once executed. The decision now comes from the same source resolution everything else uses.

WHY IT MATTERS TO THE DOMAIN

Soil temperature and moisture profiles are the headline output of Hub 1. This is the function that draws them, and it could only ever draw them from S3.

WHY IT MATTERS TO THE CODEBASE

Dead code that looks live is worse than absent code: the plan cited this local branch as evidence the rebind would be easy. It was never evidence of anything, because it had never run.

WATCH OUT FOR

That branch samples one timestep per file with `isel(time=24)`, assuming 48 half-hourly steps. It is now clamped to the file length, but this path is newly live and has had far less exercise than the code around it.

CHANGE 7 OF 8

Parameterise the stream token on the two S3 readers

data_access.py:161 · neon_notebook_wrapper.py:26

BEFORE · NEON_NOTEBOOK_WRAPPER.PY:34

```
pattern = f"{neon_site}.transient.clm2.h1.{year}*.nc"
```

AFTER · NEON_NOTEBOOK_WRAPPER.PY:39

```
stream_token: str = "h1",   # S3 copies predate the h1a/h0a rename
...
pattern = f"{neon_site}.transient.clm2.{stream_token}.{year}*.nc"
```

WHAT IT DOES, PLAINLY

The token becomes an argument, with the default left at `h1`. Behaviour is unchanged.

WHY IT MATTERS TO THE DOMAIN

No domain consequence. The S3 bucket genuinely holds old-generation files, so `h1` is the correct thing to look for there.

WHY IT MATTERS TO THE CODEBASE

Worth stating plainly: **the plan's instruction to change `h1` to `h1a` "in both files" would have broken the S3 path.** The stale-looking token was correct in this context. Only live local reads needed the new name.

WATCH OUT FOR

`neon_notebook_wrapper.py:34` was the third occurrence of this pattern and is named in neither the issue nor the plan. It is re-exported publicly from the package.

CHANGE 8 OF 8

Tests at the reader boundary

tests/test_data_access_local.py (new, 128 lines)

BEFORE · NO COVERAGE OF DATA_ACCESS.PY

```
(none)
```

AFTER · TEST_DATA_ACCESS_LOCAL.PY:107

```
def test_opens_non_empty_dataset(self):
    dataset = open_ctsm_hist("KONZ", 2018, output_root=LIVE_ROOT)
    assert dataset.sizes.get("time", 0) > 0
    assert {"TSOI", "H2OSOI", "GPP"} & set(dataset.variables)
```

WHAT IT DOES, PLAINLY

Fifteen tests. Assertions are on file counts, stream tokens, and non-zero time length rather than on calls merely not raising. Tests that need real simulation output skip when it is absent.

WHY IT MATTERS TO THE DOMAIN

Verified against actual output: KONZ 2018 gives 365 daily files and 17,520 half-hourly timesteps with TSOI, H2OSOI and GPP present, and the CLBJ 2019 reference copies read too.

WHY IT MATTERS TO THE CODEBASE

The plan specified no test coverage. Asserting non-emptiness is the whole point — a test that only checks "did not raise" would pass against an empty glob, which is the exact bug being fixed.

Cross-cutting ramifications

Two naming conventions and four archive layouts are now permanently supported, not transitionally. Phase 5 validates live output against the legacy reference copies, so both must stay readable for as long as that validation matters. Anyone "cleaning up" the probe lists will break it.

The discovery logic is now the single place that knows how output is laid out. That is the point, but it means a layout this list does not know about fails everywhere at once rather than in one caller.

Where the plan and the code disagreed

ITEM	STATUS	DETAIL
Change `h1` → `h1a` in both files	WOULD BREAK S3	S3 and reference data legitimately use `h1`. Resolved by probing instead of switching.
Engine "may be NetCDF-4"; try `h5netcdf`	BOTH WRONG	Live output is CDF-5. Neither `scipy` nor `h5netcdf` can read it; `netcdf4` can.

ITEM	STATUS	DETAIL
Live path is `{output_root}/archive/lnd/hist/`	WRONG FOR THE HUBS	`run_neon_v2.py:1086–1089` uses `{dirname(base_case_root)}/archive/{site}/{exp}/`. Data contract corrected in `ebf68fc`.
Two files need the fix	THREE	`neon_notebook_wrapper.py:34` is named nowhere in the plan or issue.
Task 4: stop requiring COS credentials	ALREADY TRUE	The library was already lazy. The eager check is a notebook cell, so Phase 2.
Local branch in the plotting helper proves the rebind is easy	DEAD CODE	Unreachable since `is_s3` was derived from a literal. Never executed.
Fixtures live in `tests/fixtures/reference_output/`	ACTUALLY ELSEWHERE	They are at `archive/archive/{CASE}/lnd/hist/`, gitignored.

Not caused by this change, but it will bite Phase 2. The container ships a baked copy of `analytics_modules` at `/opt/analytics_modules`, which shadows the repository on `sys.path`. Editing the working tree changes nothing a notebook imports unless `PYTHONPATH` points at it. This surfaced while testing.

Questions for the author

QUESTION	WHY IT NEEDS YOU
Is `/home/user` the right default for `CTSM_OUTPUT_ROOT`?	Correct in-container, requires setting on a dev host. The plan never pinned it, so this default was chosen here.
Should `run_neon_v2.py`'s forked copy of these readers be deleted rather than maintained in parallel?	It duplicates roughly 200 lines and shares the interpreter. Deleting collapses future fixes to one place, but touches the simulation driver.
Should the reference copies move to the location the plan documents?	They sit at `archive/archive/` while the plan and Phase 0 both say `tests/fixtures/reference_output/`. The probe list currently supports both.
Keep the residual hardcoded `h1` in the plotting helper's S3 branch?	`data_access.py:503` is correct for S3 but inconsistent with the parameterisation applied elsewhere.